

LAB 02 REPORT DELIVERABLES

VLSI Testing & Design for Testability

RTL Design, Cadence Genus Synthesis and QoR Analysis

Student Information

Name	R SARANG
Roll No.	EC23I2015
Design / Problem Assigned	Genus Synthesis and Fault Detection Of Seq. Multiplier
Date	2/09/2026

1. Objective

Design an RTL circuit, synthesize it using Cadence Genus, and experimentally study the effect of timing constraints on synthesis Quality of Results (QoR). The report must quantify the changes in timing, area, power, cell count, and fanout and explain the observed trade-offs.

2. Software and Tools

- Cadence Genus — RTL synthesis and QoR analysis
- Questa — RTL/gate-level functional simulation, where required
- GPDK 45 nm standard-cell library — target technology library
- Python/MATLAB/Excel — data processing and plots, if required

3. Required Files to Submit

- RTL source code (.v/.sv)
- Genus synthesis Tcl script
- SDC constraint file
- Generated gate-level netlist
- Genus QoR report
- Genus area report
- Genus timing report
- Power report, if power analysis is enabled in the supplied flow
- All frequency/constraint sweep results
- Plots generated from the experiments
- Final Lab 02 report in PDF

TASK 1:

4. Part A — RTL Design

Submit the following:

- Problem statement assigned to you.
- Functional block diagram.
- RTL source code.
- Description of all inputs and outputs.
- Clock and reset description.
- Expected functionality of the circuit.
- Simulation waveform demonstrating correct RTL functionality.

5. Part B — Baseline Cadence Genus Synthesis

Run the RTL through Cadence Genus using the initial constraints supplied/defined for the experiment.

Record the following values from the Genus reports:

Parameter	Baseline Result
Clock period	5.00 ns (200MHz)
Target frequency	200Mhz
WNS	3355.9 ps
TNS	0
Number of violating paths	0
Critical path delay	1465 ps
Combinational area	
Sequential area	
Total cell area	200.412
Total/physical area, if reported	
Combinational cell count	58
Sequential cell count	18
Total instance count	76
Maximum fanout	18
Average fanout	2.3
Dynamic power, if available	-
Leakage power, if available	-
Total power, if available	-

Attach the relevant Genus QoR, area, timing, and power reports in the appendix.

6. Part C — Frequency Sweep

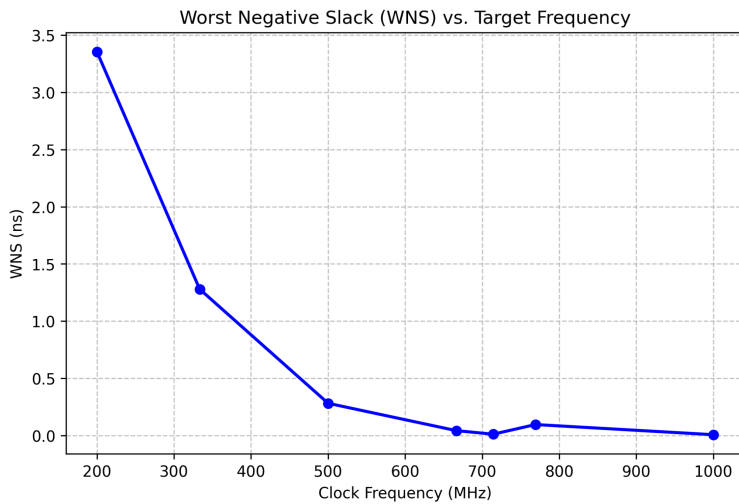
Change the clock period systematically and repeat synthesis. Use at least six different clock periods.

Choose values appropriate to your design; a suggested sweep is shown below.

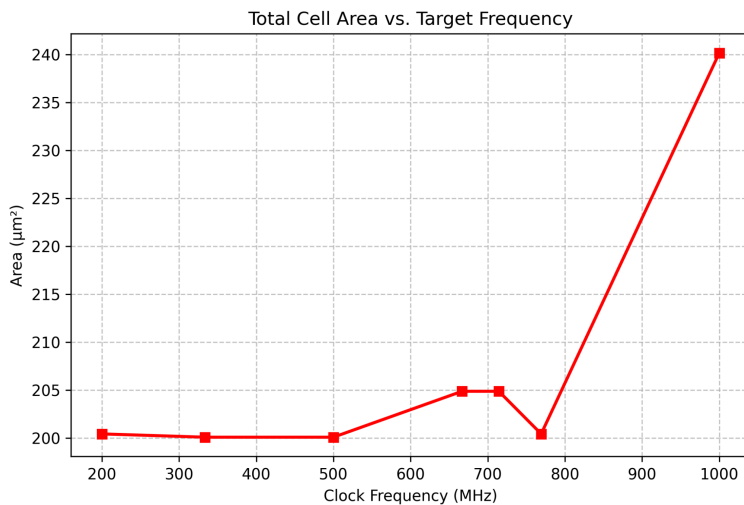
Clock Period (ns)	Frequency (MHz)	WNS (ps)	TNS (ps)	Violating Paths	Area	Power
5	200.0	3355.9	0	0	200.412	
3	333.3	1279.5	0	0	200.070	
2	500	281.3	0	0	200.070	
1.5	666.7	41.7	0	0	204.858	
1.4	714.3	11.0	0	0	204.858	
1.3	769.2	95.2	0	0	200.412	
1	1000	6.5	0	0	240.152	

Required plots:

- Clock frequency vs. WNS



- Clock frequency vs. area



- Clock frequency vs. power, if power data are available

Questions to answer:

- What happens to WNS as the target frequency is increased?
As the clock frequency goes higher and higher the WNS keeps approaching zero and then starts becoming negative.
- What happens to the area as the timing requirement becomes more aggressive?
As the target frequency increases, the total cell area remains flat until about 700 - 800 MHz and beyond this point, Genus upsizes cells and inserts extra buffers to meet the strict timing, which causes the area to increase.
- What happens to power as frequency increases?
- Does Genus change the cell selection or cell count when the timing constraint changes? Give evidence from your reports.

Until 200MHz the cell count was 76 and the critical path used small, ripple-carry adders. As the frequency increases, around 1000MHz, the cell count becomes 112 and the area goes from 200.412 to 240.152.

7. Part D — Maximum Timing-Clean Frequency

Continue reducing the clock period until the design changes from timing-clean to timing-violating. Then perform a finer sweep around this boundary.

Clock Period (ns)	Frequency (MHz)	WNS (ns)	Timing Status
2.0	500	0.2813	Passing
1.5	667	0.0417	Passing
1.3	769	0.0952	Passing
1.0	1000	0.0065	Passing
0.99	1010	-0.035	Violating
0.95	1052	-0.0435	Violating
0.9	1111	-0.0935	Violating

Determine the minimum timing-clean clock period and calculate the corresponding maximum timing-clean frequency:

$$f_{\max} = 1 / T_{\min}$$

Minimum timing-clean period: **1.0 ns**

Maximum timing-clean frequency: **1000 MHz**

WNS at this point: **0.9935 ns**

Answer:

- At what point does WNS become negative?
0.9935 ns
- What is the approximate WNS = 0 boundary?
0.9935 ns
- Why should a practical design normally include timing margin rather than operate exactly at WNS = 0?

In practical testing scenarios, operating at exactly the point where WNS becomes nil will lead to timing failures from even the slightest of environmental disturbances.

8. Part E — Input Delay Sweep

Keep the clock period fixed. Change the input delay constraint while keeping other constraints unchanged.

Input Delay (ns)	WNS (ns)	TNS (ns)	Violating Paths	Area
0.1	3.3959	0	0	200.412
0.2	3.3959	0	0	200.412
0.4	3.3962	0	0	200.412
0.6	3.3962	0	0	200.412
0.8	3.3962	0	0	200.412

1.0	3.3962	0	0	200.412
-----	--------	---	---	---------

Questions:

- How does increasing input delay affect the timing available for internal logic?
Increasing the delay decreases the timing available for internal logic. This implies the signal arrives later in the clock cycle, leaving less time for the logic to compute before the next clock edge.
- At what input delay does the design first violate timing, if applicable?
The design never violates the timing.
- Explain the trend observed in WNS.

The WNS remains constant throughout the sweep of the input delay. This tells us that the critical path in this design is entirely internal rather than originating at an input pin. As the critical path does not involve input pins, changing the input delay does not affect the overall WNS.

9. Part F — Output Delay Sweep

Keep the clock period and input delay fixed. Change the output delay constraint.

Output Delay (ns)	WNS (ns)	TNS (ns)	Violating Paths	Area
0.1	3.3959	0	0	200.412
0.2	3.3959	0	0	200.412
0.4	3.3959	0	0	200.412
0.6	3.3959	0	0	200.412
0.8	3.3964	0	0	200.412
1.0	3.3964	0	0	200.412

Questions:

- What happens to WNS as output delay increases?
The WNS remains constant throughout the output delay sweep.
- Why does output delay reduce the timing budget available to the internal circuit?
The output delay has to fit within an acceptable range so that the next IC, or the next circuit has enough time to capture it.
- Compare the input-delay and output-delay experiments.

During both the scenarios, the WNS remained constant which tells us that the critical path is completely internal and hence changing the external delays will not affect the WNS value.

10. Part G — Clock Uncertainty Sweep

Use the supplied SDC and vary clock uncertainty while keeping the other constraints fixed.

Clock Uncertainty (ns)	WNS (ns)	TNS (ns)	Violating Paths	Area
0	3.4059	0	0	200.412
0.05	3.3559	0	0	200.412
0.10	3.3059	0	0	200.412
0.20	3.2059	0	0	200.412

0.30	3.1059	0	0	200.412
------	--------	---	---	---------

Questions:

- Why does increasing clock uncertainty reduce timing margin?
Clock uncertainty acts as a direct penalty on the clock period. It forces the tool to assume the worst case scenario, meaning the data must be faster to meet the setup constraint, thus reducing the available timing margin (slack).
- Does clock uncertainty significantly change the area in your design? Support your answer with data.
There was no change in the area. The area remained exactly 200.412 for all sweeps from 0 to 0.30 ns.
- What does clock uncertainty represent physically?

It primarily represents two real-world physical imperfections: Clock Jitter (random temporal variations in the clock generator/PLL where the clock pulse is slightly longer or shorter than expected) and Clock Skew (the spatial difference in arrival time of the clock signal at two different flip-flops due to different physical wire lengths on the chip).

11. Part H — Critical Path Analysis

From the Genus timing report, identify and document the worst/critical path.

Timing Parameter	Result
Startpoint	u_datapath_working_multiplicand_reg[0]/CK
Endpoint	u_datapath_accumulated_prod_reg[7]/D
Launch clock	clk
Capture clock	clk
Data path delay	1465 ps
Arrival time	1465 ps
Required time	4820 ps
WNS	

Draw the critical path as:

u_datapath_working_multiplicand_reg[0]/CK → NAND2X1 → INVX1 → ADDFX1 → ADDFX1 → ADDFX1 → NAND4XL → XNOR2X1 → AOI22X1 → NOR2X1 → u_datapath_accumulated_prod_reg[7]/D

Answer:

- Which logic block is responsible for the critical path?
Datapath block
- Which cell or group of cells contributes significantly to the delay?
ADDFX(Full Adder) cells contribute significantly to the delay time.
- Does the critical path change when the clock constraint changes?

Yes it does.

1. u_datapath_working_multiplicand_reg[0]/CK → NAND2X1 → INVX1 → ADDFX1 → ADDFX1 → ADDFX1 → NAND4XL → XNOR2X1 → AOI22X1 → NOR2X1 → u_datapath_accumulated_prod_reg[7]/D this is the path during 5.0 ns time period.

2. u_controller_current_state_reg[0]/CK → INVX1 → NOR2X1 → NOR2X1 → AOI21X1 → OAI2BB1X1 → OAI211X1 → u_datapath_accumulated_prod_reg[6]/D this is the path during 1.0ns time period.

12. Part I — Area and Cell Usage Analysis

From the Genus area/cell reports, identify the major contributors to the area.

Cell Type	Number of Instances	Area per Cell	Total Area
DFFHQX1	13	5.472	71.136
SDFFQX1	4	7.524	30.096
ADDFX1	3	5.130	15.390
AOI22X1	6	2.052	12.312
NAND2X1	11	1.026	11.286

Answer:

- Which cell type is used most frequently?
 1. DFFHQX1
- Which cell type contributes the most area?
 1. DFFHQX1 (71.136 square micrometers)
- How many sequential cells are present?
 1. 18
- How many combinational cells are present?
 1. 58
- Does the cell composition change when a more aggressive timing constraint is used?
 1. Yes as the timing period keeps decreasing the cell count jumps from 76 to 112.

13. Part J — Fanout Analysis

Record the maximum and average fanout from the Genus QoR report.

Parameter	Result
Maximum fanout	18
Net with maximum fanout	clk
Driver of maximum-fanout net	Primary Input Port
Average fanout	2.3

Answer:

- Why can high fanout cause timing problems?
 1. Higher fanout tells us that a single gate is driving many gates which creates a higher capacitance.
- Does Genus insert buffers or change cell sizes to handle fanout in your design?
 1. Genus added buffers and upsized cells as the time period decreased from 5.0 to 1.0ns to increase the physical drive strength of the signals so they could handle the fanout load faster.

- Compare the fanout results for the relaxed and aggressive timing constraints.
 1. The maximum fanout (18) and average fanout (2.3) remained completely identical in both the constraints.

14. Part K — Relaxed vs. Aggressive Constraint Comparison

Select one relaxed timing constraint and one aggressive timing constraint from your experiments.

Metric	Relaxed Constraint	Aggressive Constraint
Clock period	5.0ns	1.0ns
Frequency	200 Mhz	1000 Mhz
WNS	3.3559 ns	0.0065 ns
TNS	0	0
Violating paths	0	0
Total area	200.412	240.152
Cell count	76	112
Maximum fanout	18	18
Total power, if available	-	-

Discussion:

Explain how and why the synthesized implementation changed when the timing requirement became more aggressive. Use evidence from cell count, cell types, area, timing, and power.

Answer:

As the timing requirements became more and more aggressive, internal restructuring was done by the tool. The evidence of this can be seen in the way the area grew from 200.412 to 240.152 units where the tool would have replaced slow half adders with bigger and more powerful cells. This optimization also caused a change in the critical path.

15. Part L — Final Design Recommendation

Based on all experiments, select the operating point you recommend for your design.

Recommended clock period: **1.3 ns**

Recommended frequency: **769.2 MHz**

WNS: **0.0952 ns**

Area: 200.412

Power:

Justify your choice using at least three QoR parameters. Do not select a frequency based only on WNS.

My recommendation is 1.3ns operating point because it offers a good balance of the trade off between area, performance. At 1.0 ns, the design

16. Important Submission Rules

- Do not submit only screenshots. Numerical results must be entered in the tables.
- Every sweep point must correspond to a separate synthesis run.
- Do not change multiple constraints simultaneously unless the experiment explicitly asks you to.
- Clearly state the technology library and operating condition used.
- Use the same RTL for all comparison experiments unless a question specifically asks for an RTL modification.
- Do not claim a maximum frequency from a single timing run; demonstrate the boundary with a sweep.
- All plots must have axis labels, units, legends where required, and meaningful titles.
- Discuss unexpected results rather than deleting them.
- Attach logs

TASK 2:

Part 1 — Exhaustive Simulation of Your Design

After completing RTL synthesis using Cadence Genus, perform exhaustive simulation of your design wherever computationally practical.

1.1 Determine the Exhaustive Test Space

Q1.1. Determine the number of independent primary-input bits in your design.

Calculate the total number of possible input combinations:

Report:

- Number of primary-input bits = 9
- Number of possible input vectors = 512
- Is exhaustive simulation practical for your design? Explain.

Yes it is practical. The number of input combinations is a low number at 512. So simulating for every individual input combination is possible and practical.

1.2 Exhaustive Simulation

Run exhaustive simulation for your design using the supplied simulation flow.

Q1.2. Generate the complete output response of the known-good circuit for all possible input combinations.

Report:

- Number of vectors simulated - 256
- Simulation time - 2 seconds
- Output/reference file generated - transcript file

- Evidence of successful completion

```
# Test: 15 x 15 | Exp Prod: 225, Act Prod: 225 | Exp Cycles: 17, Act Cycles: 17
| [ PASS ]
# Test: 15 x 0 | Exp Prod: 0, Act Prod: 0 | Exp Cycles: 2, Act Cycles: 2
| [ PASS ]
# Test: 15 x 1 | Exp Prod: 15, Act Prod: 15 | Exp Cycles: 3, Act Cycles: 3
| [ PASS ]
# Test: 15 x 2 | Exp Prod: 30, Act Prod: 30 | Exp Cycles: 4, Act Cycles: 4
| [ PASS ]
# Test: 15 x 3 | Exp Prod: 45, Act Prod: 45 | Exp Cycles: 5, Act Cycles: 5
| [ PASS ]
# Test: 15 x 4 | Exp Prod: 60, Act Prod: 60 | Exp Cycles: 6, Act Cycles: 6
| [ PASS ]
# Test: 15 x 5 | Exp Prod: 75, Act Prod: 75 | Exp Cycles: 7, Act Cycles: 7
| [ PASS ]
# Test: 15 x 6 | Exp Prod: 90, Act Prod: 90 | Exp Cycles: 8, Act Cycles: 8
| [ PASS ]
# Test: 15 x 7 | Exp Prod: 105, Act Prod: 105 | Exp Cycles: 9, Act Cycles: 9
| [ PASS ]
# Test: 15 x 8 | Exp Prod: 120, Act Prod: 120 | Exp Cycles: 10, Act Cycles: 10
| [ PASS ]
# Test: 15 x 9 | Exp Prod: 135, Act Prod: 135 | Exp Cycles: 11, Act Cycles: 11
| [ PASS ]
# Test: 15 x 10 | Exp Prod: 150, Act Prod: 150 | Exp Cycles: 12, Act Cycles: 12
| [ PASS ]
# Test: 15 x 11 | Exp Prod: 165, Act Prod: 165 | Exp Cycles: 13, Act Cycles: 13
| [ PASS ]
# Test: 15 x 12 | Exp Prod: 180, Act Prod: 180 | Exp Cycles: 14, Act Cycles: 14
| [ PASS ]
# Test: 15 x 13 | Exp Prod: 195, Act Prod: 195 | Exp Cycles: 15, Act Cycles: 15
| [ PASS ]
# Test: 15 x 14 | Exp Prod: 210, Act Prod: 210 | Exp Cycles: 16, Act Cycles: 16
| [ PASS ]
# Test: 15 x 15 | Exp Prod: 225, Act Prod: 225 | Exp Cycles: 17, Act Cycles: 17
| [ PASS ]
# =====
===
#                               TESTBENCH COMPLETE
# =====
===
# ** Note: $finish      : tb_top.v(116)
#   Time: 34620 ns  Iteration: 1  Instance: /tb_top
# End time: 23:48:39 on Sep 11,2026, Elapsed time: 0:00:02
# Errors: 0, Warnings: 1
```

Part 2 — Stuck-at Fault Injection on Your Design

Select one internal net from your **Genus synthesized gate-level netlist**.

Preferably select a net in the middle of the logic rather than a primary input/output.

2.1 Fault Selection

Q2.1. Identify your selected fault.

Report:

- Net name: n_76
- Fault type: SA-0
- Original driver cell:g4171__1881
- Fanout/load cells: g24135__6260

2.2 Fault Injection

Inject the selected stuck-at fault into your gate-level netlist.

Q2.2. Compare the original and faulty netlists.

Explain:

1. Which net was modified?
 1. n_76
2. What was the original driver?
g24171__1881
3. How was the stuck-at fault modeled?
 1. The chosen driver's original output wire was changed to a dummy wire (n_76_broken) and following that the net (n_76) was tied to ground.
4. How did the faulty netlist differ from the original netlist?
 1. The chosen driver's original output wire was changed to a dummy wire (n_76_broken) and following that the net (n_76) was tied to ground.

Include the relevant portion of both netlists.

Original:

```
AOI22X1 g24171__1881(.A0 (n_73), .A1 (n_43), .B0
(accumulated_prod[5]), .B1 (n_54), .Y (n_76));
```

Faulty:

```
AOI22X1 g24171__1881(.A0 (n_73), .A1 (n_43), .B0
(accumulated_prod[5]), .B1 (n_54), .Y (n_76_broken));
```

```
assign n_76 = 1'b0;
```

Part 3 — Fault Activation and Detection

3.1 Detecting Vector

Q3.1. Find one input vector that **detects** your selected stuck-at fault.

Report in the following format:

For input vector all 0's, the good circuit produces output n_95=0, while the faulty circuit produces output n_95=1.

Also report:

- Good-circuit value of the faulty net = 1
- Faulty-circuit value = 0
- Primary output(s) showing the error = n_95

Explain why this vector detects the fault.

The stuck-at-0 fault is successfully activated because the good circuit drives the faulty net (n_76) to 1. This fault propagates through the NAND3XL gate (g24135__6260) because its other two inputs (n_68 and n_80) are held at 1 (the non-controlling value for a NAND gate). This causes the final output of the NAND3 gate (n_95) to flip from 0 to 1, successfully detecting the fault.

Part 4 — Bit-Parallel Fault Simulation

Use the supplied bit-parallel fault simulator on your **own Genus gate-level netlist**.

For example:

```
python3 scripts/faultsim.py --selftest
```

and then:

make offline

The fault simulator should also determine a reduced set of test patterns that achieves the same fault coverage as exhaustive testing.

Parameter	Result
Total exhaustive vectors	134,217,728
Compacted test patterns	61